

판다스(Pandas) Series

인덱싱과 슬라이싱 강의 교재

광명코딩학원 | 파이썬 판다스 기초 강의

[핵심 개념] Series는 `iloc`(행 번호), `loc`(인덱스 레이블), 대괄호 `[ ]` 세 가지 방법으로 인덱싱할 수 있습니다. `iloc`와 `loc`를 명시적으로 사용하는 것을 권장합니다.

1. Series dtype (데이터 타입)

[개념] dtype은 Series에 저장된 데이터의 타입입니다. 서로 다른 타입이 섞이면 가장 범용적인 object 타입으로 통일됩니다.

dtype 예제

```
import pandas as pd

# 정수 + 문자 혼합 -> object
s = pd.Series([1, 2, 'python'])

print(s.dtype) # object

# 이유: 서로 다른 타입이 섞이면 가장 범용적인 object로 통일

# 정수만 -> int64
s2 = pd.Series([1, 2, 3])

print(s2.dtype) # int64

# 문자열만 -> object
s3 = pd.Series(['1', '2', '3'])

print(s3.dtype) # object
```

dtype	설명	예시
int64	정수형 (기본)	pd.Series([1, 2, 3])
float64	실수형	pd.Series([1.0, 2.5])
object	문자열 또는 혼합 타입	pd.Series(['a','b']) / [1,'python']
bool	불리언	pd.Series([True, False])

2. Series 타입 변경 (astype)

astype 예제

```
import pandas as pd

s = pd.Series(['1', '2', '3'])

print(s.dtype) # object (문자열)

# astype으로 int32 변경

s_int = s.astype('int32')

print(s_int.dtype) # int32

print(s_int)

# 0 1

# 1 2

# 2 3

# dtype: int32
```

[TIP] astype()은 Series의 데이터 타입을 변환하는 메서드입니다. 문자열 숫자를 정수/실수로 변환할 때 자주 사용합니다.

3. Series 데이터 타입 설정 (dtype 파라미터)

dtype 파라미터 예제

```
import pandas as pd
```

```
# 생성 시 dtype 직접 지정

data = [50000, 1, 4000]

index = ['BTC', 'XRP', 'ETH']

s = pd.Series(data, index, dtype=float)

print(s.dtype) # float64

print(s)

# BTC 50000.0

# XRP 1.0

# ETH 4000.0

# dtype: float64
```

[TIP] Series 생성 시 dtype 파라미터로 타입을 직접 지정할 수 있습니다. dtype=float, dtype=int, dtype='int32' 등 다양하게 지정 가능합니다.

4. Series 인덱싱 — 3가지 방법

방법 1. 대괄호 [] 사용 — 기본적으로 인덱스 레이블로 해석

대괄호 인덱싱

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

print(s['영희']) # 200 < - 인덱스 레이블로 접근

# 대괄호는 기본적으로 인덱스 레이블(문자)로 해석함
```

- 대괄호[]는 기본적으로 인덱스 레이블로 해석됨
- 간편하지만 판다스 버전에 따라 동작이 다를 수 있어 주의 필요

방법 2. iloc — 행 번호(정수)로 인덱싱

iloc 인덱싱

```
# iloc = integer location (정수 위치)

print(s.iloc[0]) # 100 < - 행번호 0번 (철수)

print(s.iloc[1]) # 200 < - 행번호 1번 (영희)

print(s.iloc[2]) # 300 < - 행번호 2번 (아름)

print(s.iloc[-1]) # 300 < - 마지막 값 (음수 사용 가능)
```

- iloc는 행 번호(정수)를 사용해 값을 가져옴
- 음수 인덱싱 가능: iloc[-1] = 마지막 값
- 함수가 아닌 속성이므로 () 괄호 없이 [] 대괄호 사용

방법 3. loc — 인덱스 레이블로 인덱싱

loc 인덱싱

```
# loc = location (인덱스 레이블 사용)

print(s.loc['철수']) # 100

print(s.loc['영희']) # 200
```

```
print(s.loc['아름']) # 300
```

- loc는 인덱스 레이블(문자/숫자)을 사용해 값을 가져옴
- 명시적으로 인덱스 레이블임을 표현 — 권장 방식

방법	문법	기준	특징
대괄호	s['인덱스']	인덱스 레이블	간편하나 버전별 동작 다를 수 있음
iloc	s.iloc[0]	행 번호 (정수)	명시적, 음수 인덱싱 가능, 권장
loc	s.loc['인덱스']	인덱스 레이블	명시적, 레이블 접근 권장 방식

[비교] Q4 vs Q6 vs Q7 차이: Q4(대괄호)=s['영희'], Q6(loc)=s.loc['영희'], Q7(인덱스직접)=s['영희'] — 대괄호는 내부적으로 인덱스 레이블로 해석되므로 loc와 같은 결과. 그러나 loc를 명시적으로 쓰는 것이 더 안전하고 권장됩니다.

5. Series 슬라이싱

[개념] 슬라이싱은 연속된 여러 값을 한번에 가져오는 방법입니다. iloc(행 번호 기준)는 끝 미포함, loc(인덱스 기준)는 끝 포함!

iloc 슬라이싱 — 행 번호 기준 (끝 미포함)

iloc 슬라이싱

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

print(s.iloc[0:2]) # 0번~1번 (2번 미포함)

# 철수 100

# 영희 200
```

loc 슬라이싱 — 인덱스 레이블 기준 (끝 포함!)

loc 슬라이싱

```
print(s.loc['철수':'영희']) # 철수~영희 (영희 포함!)
```

```
# 철수 100
```

```
# 영희 200
```

[주의] [중요!] iloc는 끝 인덱스 미포함 (파이썬 리스트와 동일) / loc는 끝 인덱스 포함 (인덱스 레이블 슬라이싱)

날짜 인덱스 슬라이싱 예시

날짜 슬라이싱

```
data = [100, 200, 300, 400, 500, 600]

index = ['08/01','08/02','08/03','08/04','08/05','08/06']

s = pd.Series(data, index)

# 상위 3개 날짜(인덱스) 출력

print(s.iloc[0:3].index)

# Index(['08/01', '08/02', '08/03'], dtype='object')

# 100, 300, 500 슬라이싱 (2칸 간격)

print(s.iloc[0::2])

# 08/01 100

# 08/03 300

# 08/05 500
```

6. 멀티 인덱싱 (비연속 여러 값 동시 선택)

[개념] 멀티 인덱싱은 연속적이지 않은 여러 값을 동시에 선택하는 방법입니다. 파이썬 리스트에는 없는 기능으로, 리스트를 [] 안에 넣어 사용합니다.

멀티 인덱싱

```
data = [100, 200, 300]

index = ['월요일', '화요일', '수요일']
```

```

s = pd.Series(data, index)

# iloc 멀티 인덱싱 (행 번호 리스트)

print(s.iloc[[0, 2]])

# 월요일 100

# 수요일 300

# loc 멀티 인덱싱 (인덱스 레이블 리스트)

print(s.loc[['월요일', '수요일']])

# 월요일 100

# 수요일 300

# 파이썬 리스트에서는 불가!

# m = [1, 2, 3, 4]

# m[[0, 2]] -> TypeError 발생

```

7. 주요 개념 정리

개념	설명
dtype	Series에 저장된 데이터 타입 (int64, float64, object)
astype()	Series의 데이터 타입을 변환하는 메서드. s.astype('int32')
iloc	행 번호(정수)로 인덱싱/슬라이싱. 끝 인덱스 미포함
loc	인덱스 레이블로 인덱싱/슬라이싱. 끝 인덱스 포함
대괄호[]	기본적으로 인덱스 레이블로 해석. iloc/loc 사용 권장
멀티 인덱싱	리스트로 비연속 여러 값 동시 선택. 파이썬 리스트에는 없는 기능

[실습] 실습 팁: 주피터 노트북에서 각 코드를 직접 실행해 보세요. iloc는 행 번호, loc는 인덱스 레이블 — 이 차이를 꼭 기억하세요!

판다스 Series 인덱싱/슬라이싱 연습문제

총 10문제 | 광명코딩학원

Q1. 시리즈 타입

다음 코드를 수행하면 어떤 데이터 타입이 출력되는지 이야기하세요. 그리고 그 이유는 무엇인가요?

```
import pandas as pd

s = pd.Series([1, 2, 'python'])

print(s.dtype)
```

힌트: 서로 다른 타입이 섞이면? -> s.dtype

내 답안:

Q2. 시리즈 타입 변경

다음 시리즈 객체의 데이터 타입을 int32로 변경하세요.

```
import pandas as pd

s = pd.Series(['1', '2', '3'])
```

힌트: s.astype('int32')

내 답안:

Q3. 시리즈 데이터 타입 설정

BTC 금액이 50000 달러, XRP 금액이 1 달러, ETH 금액이 4000 달러일 때 시리즈 객체로 생성하세요. 시리즈 객체를 생성할 때 가격의 데이터 타입을 float으로 지정하세요.

힌트: pd.Series(data, index, dtype=float)

내 답안:

Q4. 시리즈 인덱싱 (대괄호 사용하기)

다음 데이터를 시리즈 객체로 표현하세요. 그 다음 대괄호([])를 사용하여 200을 출력하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)
```

힌트: s['영희'] - - 대괄호는 기본적으로 인덱스 레이블로 해석

내 답안:

Q5. 시리즈 인덱싱 (iloc 속성 사용하기)

다음 데이터를 시리즈 객체로 표현하세요. 그 다음 iloc 속성을 사용하여 200을 출력하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)
```

힌트: s.iloc[1] - - iloc는 행 번호(정수) 사용

내 답안:

Q6. 시리즈 인덱싱 (loc 속성 사용하기)

다음 데이터를 시리즈 객체로 표현하세요. 그 다음 loc 속성을 사용하여 200을 출력하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)
```

힌트: s.loc['영희'] - - loc는 인덱스 레이블 사용

내 답안:

Q7. 시리즈 인덱싱 (인덱스 사용하기)

다음 데이터를 시리즈 객체로 표현하세요. 그 다음 인덱스 레이블을 직접 사용하여 200을 출력하세요. (단, s.loc 사용 금지 — 대괄호[]와 인덱스 레이블만 사용)

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)
```

힌트: s['영희'] - - 대괄호[]에 인덱스 레이블을 직접 입력

내 답안:

Q8. 시리즈 슬라이싱

다음 시리즈에서 100과 200을 시리즈로 출력하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)
```

힌트: s.iloc[0:2] 또는 s.loc['철수':'영희']

내 답안:

Q9. 시리즈 슬라이싱 (날짜 인덱스)

시리즈에는 일자별 가격 정보가 저장돼 있습니다. 시리즈에서 상위 3개 데이터의 날짜(인덱스)를 출력하세요.

```
data = [100, 200, 300, 400, 500, 600]

index = ['08/01', '08/02', '08/03', '08/04', '08/05', '08/06']

s = pd.Series(data, index)

# 기대 결과: Index(['08/01', '08/02', '08/03'], dtype='object')
```

힌트: s.iloc[0:3].index

내 답안:

Q10. 시리즈 슬라이싱 (간격)

다음 시리즈에서 100, 300, 500을 슬라이싱으로 출력하세요.

```
data = [100, 200, 300, 400, 500, 600]

index = ['08/01','08/02','08/03','08/04','08/05','08/06']

s = pd.Series(data, index)

# 기대 결과:

# 08/01 100

# 08/03 300

# 08/05 500
```

힌트: `s.iloc[0::2]` - - 2칸 간격 슬라이싱

내 답안:

정답 모음

판다스 Series 인덱싱/슬라이싱 | 광명코딩학원

Q1. 시리즈 타입	<code>print(s.dtype) # object</code> / 이유: 정수(int)와 문자열(str)이 섞여 가장 범용적인 object 타입으로 통일됨
Q2. 시리즈 타입 변경	<code>s_int = s.astype('int32') / print(s_int.dtype) # int32</code>
Q3. dtype 설정	<code>data=[50000,1,4000]; index=['BTC','XRP','ETH'] / s = pd.Series(data, index, dtype=float)</code> <code># float64</code>
Q4. 인덱싱 (대괄호)	<code>print(s['영희']) # 200</code> / 대괄호는 기본적으로 인덱스 레이블로 해석
Q5. 인덱싱 (iloc)	<code>print(s.iloc[1]) # 200</code> (행번호 1번 = 영희)
Q6. 인덱싱 (loc)	<code>print(s.loc['영희']) # 200</code>
Q7. 인덱싱 (인덱스 직접)	<code>print(s['영희']) # 200</code> / loc 없이 대괄호+인덱스 레이블 직접 사용
Q8. 슬라이싱	<code>print(s.iloc[0:2]) # 철수100, 영희200</code> / 또는: <code>print(s.loc['철수':'영희'])</code> # loc는 끝 포함
Q9. 슬라이싱 (날짜)	<code>print(s.iloc[0:3].index) / # Index(['08/01', '08/02', '08/03'], dtype='object')</code>
Q10. 슬라이싱 (간격)	<code>print(s.iloc[0::2]) / # 08/01 - > 100, 08/03 - > 300, 08/05 - > 500</code>

[완료] 수고하셨습니다! iloc(행번호)와 loc(인덱스레이블)의 차이, 슬라이싱 끝값 포함 여부를 꼭 기억하세요.
다음은 DataFrame을 배워보겠습니다.